

## Introduction

=====

```
.. toctree::  
    :maxdepth: 4
```

PORTALDOT is a new generation of decentralized Layer0-level public chain, which aims to build a high-performance global blockchain application network service system for physical industries and become the core infrastructure of Web3.0 green and trusted digital civilization.

-----

**\*\*Vision: Build a green and reliable autonomous digital civilization\*\***

The long-term vision of PORTALDOT is to build a super ecology of "blockchain +" and open the "era of great navigation" for blockchain thinking, blockchain entities and financial all-round empowerment industries. It aims to establish a new type of production relations as a DAO and develop an industrial cooperative economic community.

PORTALDOT allows for the secure transmission of any type of data between any type of blockchain protocols, unlocking a vast array of real-world application scenarios. By combining the best features from multiple specialized blockchains, PORTALDOT paves the way for a new decentralized market and provides more equitable service support for a wide range of decentralized applications.

**\*\*Technical architecture: dynamic heterogeneous fragmentation and 10,000-level TPS\*\***

The underlying architecture of PORTALDOT is based on the DHSA. Like a sophisticated neural network, it intelligently adjusts the computing power of 256 shards through AI-driven resource scheduling algorithms in a rapidly changing environment.

This design overcomes the limitations of traditional fragmented blockchain technology, making 10,000-level TPS and sub-second-level confirmations a reality. It also paves a new path for mapping physical assets to blockchains.

Under the framework of the RWA standardized platform, traditional assets such as real estate, commodities, and intellectual property

can be seamlessly converted into programmable digital credentials. Its trusted on-chain ownership mechanism is reshaping the fundamental rules for global value circulation.

#### **\*\*Privacy and Security: ZKP and Quantum-Resistant Encryption\*\***

In order to ensure privacy and security, PORTALDOT cleverly utilizes ZKP technology to build a dual-encrypted network, ensuring the security of all transactions.

The anti-quantum encryption algorithm supported by the top cryptography laboratory is a customized solution for the optimization of the unique attributes of the blockchain network, with deep and long-term "quantum thinking".

#### **\*\*AI+PORTALDOT : Self-evolution of Smart contracts\*\***

PORTALDOT's built-in distributed machine learning framework integrates homomorphic encryption and federated learning technologies, enabling sensitive information such as medical data and financial transactions to participate in model training without decryption.

This not only fully unlocks the value of data elements but also injects autonomous evolutionary capabilities into smart contract development. PORTALDOT's AI-driven contract optimization engine monitors and protects against 32 types of security vulnerabilities, including reentrancy attacks, in real time, and utilizes reinforcement learning to dynamically adjust gas consumption models.

#### **\*\*Consensus Mechanism: LAO NPoS\*\***

PORTALDOT pioneered and implemented the LAO NPoS (Linear Attenuation Offset Nominated Proof of Stake) consensus mechanism. This innovative consensus algorithm, through a reward halving mechanism, promotes dynamic economic model optimization in an algorithm-driven manner, guiding the steady growth of the ecosystem.

According to the staking ratio of the whole network, LAO NPoS realizes the actual dynamic inflation reduction model of PORTALDOT, directly improves the scarcity attribute of POT, and promotes its evolution from "functional token" to "value storage medium".

By suppressing early rapid inflation, this mechanism automatically reduces market pressure, attracts ecological builders and institutional investors, and optimizes the structure of ecological participants.

## **\*\*Developer-Ecosystem:Privacy Modularization and Cross-Chain Interoperability\*\***

IN terms of developer ecological construction, PORTALDOT shows a higher pattern. The modular smart contract library supports rapid combination innovation, and the cross-chain API gateway eliminates the disconnection of the multi-chain ecosystem, enabling developers to make more efficient use of underlying technologies.

This design concept of "eliminating complexity from the bottom layer and returning simplicity to developers" has attracted a group of innovative forces that want to break the constraints of traditional frameworks. Developers can use the SDK to quickly build customized multi-signature DApps and share the benefits of PORTALDOT's cryptographic infrastructure.

## **\*\*Future: Value operating system of Web3.0\*\***

With the proposal and realization of the LAO NPoS consensus and the upgrading of the account system and staking model, PORTALDOT has completed the evolution from "cross-chain protocol" to "Web3.0 value operating system". This system-level technological breakthrough enables PORTALDOT to support the value exchange needs of complex scenarios such as social finance, supply chain digitalization and open science in the form of "protocol clusters" rather than a single public chain.

In the future, PORTALDOT may integrate modules such as self-sovereign identity (SSI) and data availability layer (DA) to build a complete decentralized protocol stack, and attract more privacy-centric computing and artificial intelligence projects to join through developer incentive programs to form a cross-chain smart contract network aggregate.

The enterprise-level account system paves the way for the chain of compliant stablecoins and RWA assets, and breaks through the last barrier between DeFi and traditional finance. PORTALDOT is building a green and credible autonomous digital civilization system that has strong autonomous technical sovereignty and does require trust intermediaries.

## **Local Development Network**

=====

```
.. toctree::  
    :maxdepth: 4
```

You can easily start a local development network by next steps.

-----

## **\*\*Download client\*\***

Download the client refer to :doc:`../chain-info`.

Taking the Ubuntu client as an example, we decompress the downloaded file  
``portaldot-testnet-ubuntu.tar.gz``

and then execute the following commands.

```
.. code:: bash
```

```
tar -xzf portaldot-testnet-ubuntu.tar.gz
cd portaldot-testnet-ubuntu
chmod 755 portaldot_dev
```

## **\*\*Run a local development node\*\***

```
.. code:: bash
```

```
portaldot_dev --dev --alice
```

## **\*\*Connect to chain explorer\*\***

Open an web browser (e.g., Chrome) and go to the `Portaldot explorer  
<<https://www.portaldot.io/>>`\_.

Select the ``Local Node`` in the network options.

```
.. image:: network-switch.png
   :align: center
```

## Chain Info

=====

```
.. toctree::
   :maxdepth: 4
```

## **\*\*Chain Info\*\***

```
=====
Key      Value
=====
websocket  wss://mainnet.portaldot.io
ss58_format 42
=====
```

## **\*\*Token Info\*\***

```
=====
Key      Value
=====
name     POT
decimal  14
=====
```

## **\*\*Portaldot mainnet node client\*\***

```
=====
OS      Link
=====
Ubuntu  `Download
<https://github.com/portaldotVolunteer/Portaldot-node/raw/main/portaldot-mainnet-ubuntu.tar.gz
>`_
MacOS   `Download
<https://github.com/portaldotVolunteer/Portaldot-node/raw/main/portaldot-mainnet-macos.tar.gz
>`_
Windows recommended to run in `WSL <https://learn.microsoft.com/en-us/windows/wsl/>`_
=====
```

## **\*\*Portaldot local development node client\*\***

```
=====
OS      Link
=====
Ubuntu  `Download
<https://github.com/portaldotVolunteer/Portaldot-node/raw/main/portaldot-testnet-ubuntu.tar.gz
>`_
MacOS   `Download
<https://github.com/portaldotVolunteer/Portaldot-node/raw/main/portaldot-testnet-macos.tar.gz>`
_
```

Windows recommended to run in `WSL <<https://learn.microsoft.com/en-us/windows/wsl/>>` \_  
=====

Python SDK  
=====

```
.. toctree::  
    :maxdepth: 4
```

Install  
Examples  
usage/usage  
Reference/modules

Install  
=====

```
.. toctree::  
    :maxdepth: 4
```

Install using PyPI  
-----

```
.. code:: bash
```

```
# bash  
pip install substrate-interface
```

Initialization  
-----

```
.. code:: python
```

```
portaldot = SubstrateInterface(  
    url="wss://mainnet.portaldot.io",  
    ss58_format=42,  
    type_registry_preset='default'  
)
```

If connecting to a local node, ``url`` should be setting as

.. code:: python

```
url="ws://127.0.0.1:9944"
```

Quick usage

-----

**\*\*Balance information of an account\*\***

.. code:: python

```
result = portaldot.query('System', 'Account',
['5E9oDs9PjpsBbxXxRE9uMaZZhnBAV38n2ouLB28oecBDdeQo'])
print(result.value['data']['free']) # 635278638077956496
```

**\*\*Create balance transfer extrinsic\*\***

.. code:: python

```
call = portaldot.compose_call(
    call_module='Balances',
    call_function='transfer_keep_alive',
    call_params={
        'dest': '5E9oDs9PjpsBbxXxRE9uMaZZhnBAV38n2ouLB28oecBDdeQo',
        'value': 1 * 10**12
    }
)
keypair = Keypair.create_from_uri('///Alice')
extrinsic = portaldot.create_signed_extrinsic(call=call, keypair=keypair)
receipt = portaldot.submit_extrinsic(extrinsic, wait_for_inclusion=True)

print(f"Extrinsic '{receipt.extrinsic_hash}' sent and included in block '{receipt.block_hash}'")
```

Examples

=====

.. toctree::

    :maxdepth: 4

## Batch call

.. code:: python

```
from substrateinterface import SubstrateInterface, Keypair
from substrateinterface.exceptions import SubstrateRequestException

portaldot = SubstrateInterface(
    url="wss://mainnet.portaldot.io",
    ss58_format=42,
    type_registry_preset='default'
)

keypair = Keypair.create_from_uri('//Alice')

balance_call = portaldot.compose_call(
    call_module='Balances',
    call_function='transfer_keep_alive',
    call_params={
        'dest': '5FHneW46xGXgs5mUiveU4sbTyGBzmstUspZC92UhjJM694ty',
        'value': 1 * 10 ** 14
    }
)

call = portaldot.compose_call(
    call_module='Utility',
    call_function='batch',
    call_params={
        'calls': [balance_call, balance_call]
    }
)

extrinsic = portaldot.create_signed_extrinsic(
    call=call,
    keypair=keypair,
    era={'period': 64}
)

try:
    receipt = portaldot.submit_extrinsic(extrinsic, wait_for_inclusion=True)

    print('Extrinsic "{}" included in block "{}".format(
```



```

        receipt.extrinsic_hash, receipt.block_hash
    ))

    if receipt.is_success:

        print('Success, triggered events:')
        for event in receipt.triggered_events:
            print(f'* {event.value}')

    else:
        print('Extrinsic Failed: ', receipt.error_message)

```

```

except SubstrateRequestException as e:
    print("Failed to send: {}".format(e))

```

Fee info  
-----

.. code:: python

```

from substrateinterface import SubstrateInterface, Keypair

portaldot = SubstrateInterface(
    url="wss://mainnet.portaldot.io",
    ss58_format=42,
    type_registry_preset='default'
)

keypair = Keypair.create_from_uri('///Alice')

call = portaldot.compose_call(
    call_module='Balances',
    call_function='transfer_keep_alive',
    call_params={
        'dest': '5FHneW46xGXgs5mUiveU4sbTyGBzmstUspZC92UhjJM694ty',
        'value': 1 * 10 ** 14
    }
)

# Get payment info
payment_info = portaldot.get_payment_info(call=call, keypair=keypair)

print("Payment info: ", payment_info)

```

## Query a Mapped storage function

---

.. code:: python

```
from substrateinterface import SubstrateInterface

portaldot = SubstrateInterface(
    url="wss://mainnet.portaldot.io",
    ss58_format=42,
    type_registry_preset='default'
)

result = portaldot.query_map("System", "Account", max_results=100)

for account, account_info in result:
    print(f'{account.value}: {account_info.value}')
```

## Multisig transaction

---

.. code:: python

```
from substrateinterface import SubstrateInterface, Keypair

portaldot = SubstrateInterface(
    url="wss://mainnet.portaldot.io",
    ss58_format=42,
    type_registry_preset='default'
)

keypair_alice = Keypair.create_from_uri('//Alice', ss58_format=portaldot.ss58_format)
keypair_bob = Keypair.create_from_uri('//Bob', ss58_format=portaldot.ss58_format)
keypair_charlie = Keypair.create_from_uri('//Charlie', ss58_format=portaldot.ss58_format)

# Generate multi-sig account from signatories and threshold
multisig_account = portaldot.generate_multisig_account(
    signatories=[
        keypair_alice.ss58_address,
        keypair_bob.ss58_address,
        keypair_charlie.ss58_address
    ],
    threshold=2
)
```

```

)

call = portaldot.compose_call(
    call_module='Balances',
    call_function='transfer_keep_alive',
    call_params={
        'dest': keypair_alice.ss58_address,
        'value': 3 * 10 ** 14
    }
)

# Initiate multisig tx
extrinsic = portaldot.create_multisig_extrinsic(call, keypair_alice, multisig_account,
era={'period': 64})

receipt = portaldot.submit_extrinsic(extrinsic, wait_for_inclusion=True)

if not receipt.is_success:
    print(f"{receipt.error_message}")
    exit()

# Finalize multisig tx with other signatory
extrinsic = portaldot.create_multisig_extrinsic(call, keypair_bob, multisig_account,
era={'period': 64})

receipt = portaldot.submit_extrinsic(extrinsic, wait_for_inclusion=True)

if receipt.is_success:
    print(f"{receipt.triggered_events}")
else:
    print(f"{receipt.error_message}")

```

Create and call ink! contract

-----  
.. code:: python

```
import os
```

```
from substrateinterface.contracts import ContractCode, ContractInstance
from substrateinterface import SubstrateInterface, Keypair
```

```
portaldot = SubstrateInterface(
    url="wss://mainnet.portaldot.io",
```

```

    ss58_format=42,
    type_registry_preset='default'
)

keypair = Keypair.create_from_uri('///Alice')
contract_address = "5GhwarrVMH8kjb8XyW6zCfURHbHy3v84afzLbADyYYX6H2Kk"

# Check if contract is on chain
contract_info = portaldot.query("Contracts", "ContractInfoOf", [contract_address])

if contract_info.value:

    print(f'Found contract on chain: {contract_info.value}')

    # Create contract instance from deterministic address
    contract = ContractInstance.create_from_address(
        contract_address=contract_address,
        metadata_file=os.path.join(os.path.dirname(__file__), 'assets', 'flipper.json'),
        portaldot=portaldot
    )
else:

    # Upload WASM code
    code = ContractCode.create_from_contract_files(
        metadata_file=os.path.join(os.path.dirname(__file__), 'assets', 'flipper.json'),
        wasm_file=os.path.join(os.path.dirname(__file__), 'assets', 'flipper.wasm'),
        portaldot=portaldot
    )

    # Deploy contract
    print('Deploy contract...')
    contract = code.deploy(
        keypair=keypair,
        constructor="new",
        args={'init_value': True},
        value=0,
        gas_limit={'ref_time': 25990000000, 'proof_size': 11990383647911208550},
        upload_code=True
    )

    print(f'Deployed @ {contract.contract_address}')

# Read current value
result = contract.read(keypair, 'get')
```

```

print('Current value of "get":', result.contract_result_data)

# Do a gas estimation of the message
gas_predit_result = contract.read(keypair, 'flip')

print('Result of dry-run: ', gas_predit_result.value)
print('Gas estimate: ', gas_predit_result.gas_required)

# Do the actual call
print('Executing contract call...')
contract_receipt = contract.exec(keypair, 'flip', args={

}, gas_limit=gas_predit_result.gas_required)

if contract_receipt.is_success:
    print(f'Events triggered in contract: {contract_receipt.contract_events}')
else:
    print(f'Error message: {contract_receipt.error_message}')

result = contract.read(keypair, 'get')

print('Current value of "get":', result.contract_result_data)

```

Historic balance

-----

.. code:: python

```

from substrateinterface import SubstrateInterface

portaldot = SubstrateInterface(
    url="wss://mainnet.portaldot.io",
    ss58_format=42,
    type_registry_preset='default'
)

block_number = 10
block_hash = portaldot.get_block_hash(block_number)

result = portaldot.query(
    "System", "Account", ["5GrwvaEF5zXb26Fz9rcQpDWS57CtERHpNehXCPcNoHGKutQY"],
    block_hash=block_hash
)

```

```
def format_balance(amount: int):
    amount = format(amount / 10**portaldot.properties.get('tokenDecimals', 0), ".15g")
    return f"{{amount}} {{portaldot.properties.get('tokenSymbol', 'UNIT')}}"
```

```
balance = (result.value["data"]["free"] + result.value["data"]["reserved"])
```

```
print(f"Balance @ {block_number}: {format_balance(balance)}")
```

## Block headers subscription

-----

.. code:: python

```
from substrateinterface import SubstrateInterface
```

```
portaldot = SubstrateInterface(
    url="wss://mainnet.portaldot.io",
    ss58_format=42,
    type_registry_preset='default'
)
```

```
def subscription_handler(obj, update_nr, subscription_id):
    print(f"New block #{obj['header']['number']}")
```

```
    block = portaldot.get_block(block_number=obj['header']['number'])
```

```
    for idx, extrinsic in enumerate(block['extrinsics']):
        print(f"# {idx}: {extrinsic.value}")
```

```
    if update_nr > 2:
        return {'message': 'Subscription will cancel when a value is returned',
            'updates_processed': update_nr}
```

```
result = portaldot.subscribe_block_headers(subscription_handler)
print(result)
```

## Storage subscription

-----

.. code:: python

```

from substrateinterface import SubstrateInterface

portaldot = SubstrateInterface(
    url="wss://mainnet.portaldot.io",
    ss58_format=42,
    type_registry_preset='default'
)

def subscription_handler(account_info_obj, update_nr, subscription_id):

    if update_nr == 0:
        print('Initial account data:', account_info_obj.value)

    if update_nr > 0:
        # Do something with the update
        print('Account data changed:', account_info_obj.value)

    # The execution will block until an arbitrary value is returned, which will be the result of the
    `query`
    if update_nr > 5:
        return account_info_obj

result = portaldot.query("System", "Account",
["5GrwvaEF5zXb26Fz9rcQpDWS57CtERHpNehXCPcNoHGKutQY"],
    subscription_handler=subscription_handler)

print(result)

```

Subscribe to multiple storage keys

-----

.. code:: python

```

from substrateinterface import SubstrateInterface

def subscription_handler(storage_key, updated_obj, update_nr, subscription_id):
    print(f"Update for {storage_key.params[0]}: {updated_obj.value}")

portaldot = SubstrateInterface(

```

```

    url="wss://mainnet.portaldot.io",
    ss58_format=42,
    type_registry_preset='default'
)

# Accounts to track
storage_keys = [
    portaldot.create_storage_key(
        "System", "Account",
        ["5GrwvaEF5zXb26Fz9rcQpDWS57CtERHpNehXCPcNoHGKutQY"]
    ),
    portaldot.create_storage_key(
        "System", "Account", ["5FHneW46xGXgs5mUiveU4sbTyGBzmstUspZC92UhjJM694ty"]
    )
]

result = portaldot.subscribe_storage(
    storage_keys=storage_keys, subscription_handler=subscription_handler
)

```

## Keypair creation and signing

=====

```

.. toctree::
    :maxdepth: 4

```

Keypairs are used to sign transactions and encrypt/decrypt messages. They consist of a public/private key and can be generated in several ways like by a BIP39 mnemonic:

```

.. code:: python

```

```

mnemonic = Keypair.generate_mnemonic()
keypair = Keypair.create_from_mnemonic(mnemonic)
signature = keypair.sign("Test123")
if keypair.verify("Test123", signature):
    print("Verified")

```

By default, a keypair is using

`SR25519` <<https://research.web3.foundation/en/latest/polkadot/keys/1-accounts-more.html>> cryptography, alternatively ED25519 and ECDSA (for Ethereum-style addresses) can be explicitly specified:

```

.. code:: python

```



```
keypair = Keypair.create_from_mnemonic(mnemonic, crypto_type=KeypairType.ECDSA)
print(keypair.ss58_address)
# '0x6741864968e8b87c6e32e19cde88A11a3Cc636E9'
```

**\*\*Creating keypairs with soft and hard key derivation paths\*\***

.. code:: python

```
mnemonic = Keypair.generate_mnemonic()
keypair = Keypair.create_from_uri(mnemonic + '//hard/soft')
```

By omitting the mnemonic the default development mnemonic is used:

.. code:: python

```
keypair = Keypair.create_from_uri('//Alice')
```

**\*\*Creating ECDSA keypairs with BIP44 derivation paths\*\***

.. code:: python

```
mnemonic = Keypair.generate_mnemonic()
keypair = Keypair.create_from_uri(f'{{mnemonic}}/m/44'/60'/0'/0/0",
crypto_type=KeypairType.ECDSA)
```

**\*\*Create Keypair from PolkadotJS JSON format\*\***

.. code:: python

```
with open('5GrwvaEF5zXb26Fz9rcQpDWS57CtERHPNehXCPcNoHGKutQY.json', 'r') as fp:
    json_data = fp.read()
    keypair = Keypair.create_from_encrypted_json(json_data, passphrase="test",
ss58_format=42)
```

**\*\*Verify generated signature with public address\*\***

**\*Example: Substrate style addresses\***

.. code:: python

```
keypair = Keypair.create_from_uri("//Alice", crypto_type=KeypairType.SR25519)
signature = keypair.sign('test')
```

```
keypair_public =
Keypair(ss58_address='5GrwvaEF5zXb26Fz9rcQpDWS57CtERHpNehXCPcNoHGKutQY',
crypto_type=KeypairType.SR25519)
result = keypair_public.verify('test', signature)
```

\*Example: Ethereum style addresses\*

.. code:: python

```
keypair = Keypair.create_from_uri("/m/44'/60'/0'/0", crypto_type=KeypairType.ECDSA)
signature = keypair.sign('test')

keypair_public = Keypair(public_key='0x5e20a619338338772e97aa444e001043da96a43b',
crypto_type=KeypairType.ECDSA)
result = keypair_public.verify('test', signature)
```

**\*\*Offline signing of extrinsics\*\***

This example generates a signature payload which can be signed on another (offline) machine and later on sent to the network with the generated signature.

1. Generate signature payload on online machine:

.. code:: python

```
portaldot = SubstrateInterface(
    url="wss://mainnet.portaldot.io",
    ss58_format=42,
    type_registry_preset='default'
)

call = portaldot.compose_call(
    call_module='Balances',
    call_function='transfer_keep_alive',
    call_params={
        'dest': '5GrwvaEF5zXb26Fz9rcQpDWS57CtERHpNehXCPcNoHGKutQY',
        'value': 2 * 10**8
    }
)

era = {'period': 64, 'current': 22719}
nonce = 0
```

```
signature_payload = portaldot.generate_signature_payload(call=call, era=era, nonce=nonce)
```

2. Then on another (offline) machine generate the signature with given

```
``signature_payload``:
```

```
.. code:: python
```

```
keypair = Keypair.create_from_mnemonic("nature exchange gasp toy result bacon coin  
broccoli rule oyster believe lyrics")  
signature = keypair.sign(signature_payload)
```

3. Finally on the online machine send the extrinsic with generated  
signature:

```
.. code:: python
```

```
keypair =  
Keypair(ss58_address="5EChUec3ZQhUvY1g52ZbfBVkqjUY9Kcr6mcEvQMbmd38shQL")
```

```
extrinsic = portaldot.create_signed_extrinsic(  
    call=call,  
    keypair=keypair,  
    era=era,  
    nonce=nonce,  
    signature=signature  
)
```

```
result = portaldot.submit_extrinsic(  
    extrinsic=extrinsic  
)
```

```
print(result.extrinsic_hash)
```

Query storage

=====

```
.. toctree::  
    :maxdepth: 4
```

In Portaldot, any pallet can introduce new storage items that will become part of the blockchain state. These storage items can be simple single values, or more complex storage maps.

## Example

.. code:: python

```
result = portaldot.query('System', 'Account',
['F4xQKRUagnSGjFqafyhajLs94e7Vvzvr8ebwYJceKpr8R7T'])

print(result.value['nonce'])
print(result.value['data']['free'])
```

**\*\*State at a specific block hash\*\***

.. code:: python

```
account_info = portaldot.query(
    module='System',
    storage_function='Account',
    params=['F4xQKRUagnSGjFqafyhajLs94e7Vvzvr8ebwYJceKpr8R7T'],

block_hash='0x176e064454388fd78941a0bace38db424e71db9d5d5ed0272ead7003a02234fa'
)

print(account_info['nonce'].value)
print(account_info['data']['free'].value)
```

**\*\*Type decomposition information\*\***

Some storage functions need parameters and some of those parameter types can be quite complex to compose.

To retrieve more information how to format those storage function parameters, the helper function ``get\_param\_info()`` is available:

.. code:: python

```
storage_function = portaldot.get_metadata_storage_function("Tokens", "TotalIssuance")

print(storage_function.get_param_info())
```

**\*\*Querying multiple storage entries at once\*\***

When a large amount of storage entries is requested, the most efficient way is to use the ``query\_multi()`` function. This will batch all the requested storage entries in one RPC request.

.. code:: python

```
storage_keys = [
    portaldot.create_storage_key(
        "System", "Account", ["F4xQKRUagnSGjFqafyhajLs94e7Vvzvr8ebwYJceKpr8R7T"]
    ),
    portaldot.create_storage_key(
        "System", "Account", ["GSEX8kR4Kz5UZGhvRUCJG93D5hhTAoVZ5tAe6Zne7V42DSi"]
    ),
    portaldot.create_storage_key(
        "Staking", "Bonded", ["GSEX8kR4Kz5UZGhvRUCJG93D5hhTAoVZ5tAe6Zne7V42DSi"]
    )
]
```

```
result = portaldot.query_multi(storage_keys)
```

```
for storage_key, value_obj in result:
    print(storage_key, value_obj)
```

**\*\*Query a mapped storage function\*\***

Mapped storage functions can be iterated over all key/value pairs, for these type of storage functions ``query\_map()`` can be used.

The result is a ``QueryMapResult`` object, which is an iterator:

.. code:: python

```
# Retrieve the first 199 System.Account entries
result = portaldot.query_map('System', 'Account', max_results=199)

for account, account_info in result:
    print(f"Free balance of account '{account.value}': {account_info.value['data']['free']}")
```

These results are transparently retrieved in batches capped by the ``page\_size`` kwarg, currently the maximum ``page\_size`` restricted by the RPC node is 1000

.. code:: python

```
# Retrieve all System.Account entries in batches of 200 (automatically appended by
`QueryMapResult` iterator)
result = portaldot.query_map('System', 'Account', page_size=200, max_results=400)
```

```

for account, account_info in result:
    print(f"Free balance of account '{account.value}': {account_info.value['data']['free']}")

```

Querying a ``DoubleMap`` storage function:

.. code:: python

```

era_stakers = portaldot.query_map(
    module='Staking',
    storage_function='ErasStakers',
    params=[2100]
)

```

Extrinsics

=====

```

.. toctree::
    :maxdepth: 4

```

In Portaldot, transactions are often more broadly referred to as (signed) extrinsics. The term extrinsic is generally used to mean any information that originates outside of the runtime. An extrinsic is basically a vehicle that carries the intention to execute a function call in the runtime, along with proof of the account that wants to execute it.

**\*\*Creating extrinsics\*\***

**\*Example:\***

.. code:: python

```

call = portaldot.compose_call(
    call_module='Balances',
    call_function='transfer_keep_alive',
    call_params={
        'dest': '5E9oDs9PjpsBbxXxRE9uMaZZhnBAV38n2ouLB28oecBDdeQo',
        'value': 1 * 10** 14
    }
)

```

```
extrinsic = portaldot.create_signed_extrinsic(call=call, keypair=keypair)
```

```
try:
```

```
    receipt = portaldot.submit_extrinsic(extrinsic, wait_for_inclusion=True)
```

```
    print(f"Extrinsic '{receipt.extrinsic_hash}' sent and included in block '{receipt.block_hash}'")
```

```
except SubstrateRequestException as e:
```

```
    print("Failed to send: {}".format(e))
```

The ``wait\_for\_inclusion`` keyword argument used in the example above will block giving the result until it gets confirmation from the node that the extrinsic is successfully included in a block. The ``wait\_for\_finalization`` keyword will wait until extrinsic is finalized. Note this feature is only available for websocket connections.

## **\*\*Extrinsic Receipts\*\***

The ``portaldot.submit\_extrinsic()`` example above returns an ``ExtrinsicReceipt`` object, which contains information about the on-chain execution of the extrinsic. Because the ``block\_hash`` is necessary to retrieve the triggered events from storage, most information is only available when ``wait\_for\_inclusion=True`` or ``wait\_for\_finalization=True`` is used when submitting an extrinsic.

Examples:

.. code:: python

```
receipt = portaldot.submit_extrinsic(extrinsic, wait_for_inclusion=True)
print(receipt.is_success) # False
print(receipt.weight)
print(receipt.total_fee_amount)
print(receipt.error_message['name']) # 'LiquidityRestrictions'
```

``ExtrinsicReceipt`` objects can also be created for all existing extrinsics on-chain:

.. code:: python

```
receipt = portaldot.retrieve_extrinsic_by_identifier("5233297-1")
```

```

print(receipt.is_success)
print(receipt.extrinsic.call_module.name)
print(receipt.extrinsic.call.name)
print(receipt.weight)
print(receipt.total_fee_amount)
print(receipt.error_message['docs'])

for event in receipt.triggered_events:
    print(f'* {event.value}')
```

**\*\*Multisig extrinsics\*\***

Substrate has the functionality for multi-signature dispatch, allowing multiple signed origins (accounts) to coordinate and dispatch a call, derivable deterministically from the set of account IDs and the threshold number of accounts from the set that must approve it.

To initiate and finalize multisig extrinsics, the following helper functions are available:

\*Define the multisig account by supplying its signatories and threshold:\*

.. code:: python

```

keypair_alice = Keypair.create_from_uri('///Alice', ss58_format=portaldot.ss58_format)

multisig_account = portaldot.generate_multisig_account(
    signatories=[
        keypair_alice.ss58_address,
        '5FHneW46xGXgs5mUiveU4sbTyGBzmstUspZC92UhjJM694ty',
        '5FLSigC9HGRKVhB9FiEo4Y3koPsNmBmLJbpXg2mp1hXcS59Y'
    ],
    threshold=2
)
```

\*Then initiate the multisig extrinsic by providing the call and a keypair of one of its signatories:\*

.. code:: python

```

call = portaldot.compose_call(
    call_module='System',
    call_function='remark_with_event',
```



```

    call_params={
        'remark': 'Multisig test'
    }
)

```

```

extrinsic = portaldot.create_multisig_extrinsic(call, keypair_alice, multisig_account,
era={'period': 64})
receipt = portaldot.submit_extrinsic(extrinsic, wait_for_inclusion=True)

```

\*Then a second signatory approves and finalizes the call by providing the same call to another multisig extrinsic:\*

.. code:: python

```

# Define the multisig account by supplying its signatories and threshold
keypair_charlie = Keypair.create_from_uri('//Charlie', ss58_format=portaldot.ss58_format)

```

```

multisig_account = portaldot.generate_multisig_account(
    signatories=[
        keypair_charlie.ss58_address,
        '5GrwvaEF5zXb26Fz9rcQpDWS57CtERHpNehXCPcNoHGKutQY',
        '5FLSigC9HGRKVhB9FiEo4Y3koPsNmBmLJbpXg2mp1hXcS59Y'
    ],
    threshold=2
)

```

```

extrinsic = portaldot.create_multisig_extrinsic(call, keypair_charlie, multisig_account,
era={'period': 64})
receipt = portaldot.submit_extrinsic(extrinsic, wait_for_inclusion=True)

```

The call will be executed when the second and final multisig extrinsic is submitted, condition and state of the multig will be checked on-chain during processing of the multisig extrinsic.

**\*\*Type decomposition of call params\*\***

The structure of certain call parameters can be quite complex, then the ``get\_param\_info()`` function of the call function object can provide more insight how to construct those parameters:

.. code:: python

```

call_function = portaldot.get_metadata_call_function("POT", "transfer")
param_info = call_function.get_param_info()

```

**\*\*Estimate of network fees\*\***

.. code:: python

```
payment_info = portaldot.get_payment_info(call=call, keypair=keypair)
```

**\*\*Mortal extrinsics\*\***

By default, *immortal* extrinsics are created, which means they have an indefinite lifetime for being included in a block. However, it is recommended to use specify an expiry window, so you know after a certain amount of time if the extrinsic is not included in a block, it will be invalidated.

.. code:: python

```
extrinsic = portaldot.create_signed_extrinsic(call=call, keypair=keypair, era={'period': 64})
```

The `period` specifies the number of blocks the extrinsic is valid counted from current head.

Call runtime APIs

=====

.. toctree::  
:maxdepth: 4

A Portaldot node contains a runtime. The runtime contains the business logic of the chain. It defines what transactions are valid and invalid and determines how the chain's state changes in response to transactions.

**\*\*Example\*\***

.. code:: python

```
result = portaldot.runtime_call("AccountNonceApi", "account_nonce",  
["5GrwvaEF5zXb26Fz9rcQpDWS57CtERHpNehXCPcNoHGKutQY"])  
# <U32(value=2)>
```

**\*\*List of available runtime APIs and methods\*\***

.. code:: python

```
runtime_calls = portaldot.get_metadata_runtime_call_functions()
#[
#   <RuntimeCallDefinition(value={'description': 'The API to query account nonce (aka
transaction index)', 'params': [{'name': 'account_id', 'type': 'AccountId'}], 'type': 'Index', 'api':
'AccountNonceApi', 'method': 'account_nonce'})>
#   ...
#]
```

**\*\*Get param type decomposition\*\***

A helper function to compose the parameters for this runtime API call

.. code:: python

```
runtime_call = portaldot.get_metadata_runtime_call_function("ContractsApi", "call")
param_info = runtime_call.get_param_info()
```

Cleanup and context manager

=====

```
.. toctree::
    :maxdepth: 4
```

At the end of the lifecycle of a ``SubstrateInterface`` instance, calling the ``close()`` method will do all the necessary cleanup, like closing the websocket connection.

When using the context manager this will be done automatically:

.. code:: python

```
with SubstrateInterface(url="wss://rpc.polkadot.io") as portaldot:
    events = portaldot.query("System", "Events")
```

ink! contract interfacing

=====

```
.. toctree::
```

:maxdepth: 4

ink! is a programming language for smart contracts; blockchains built with the Substrate framework can choose from a number of smart contract languages which one(s) they want to support. ink! is one of them. It is an opinionated language that we have built by extending the popular Rust programming language with functionality needed to make it smart contract compatible.

`More information about ink! <<https://use.ink/>>`\_\_

**\*\*Deploy a contract\*\***

.. code:: python

```
portaldot = SubstrateInterface(
    url="wss://mainnet.portaldot.io",
    ss58_format=42,
    type_registry_preset='default'
)

keypair = Keypair.create_from_uri('///Alice')

# Deploy contract
code = ContractCode.create_from_contract_files(
    metadata_file=os.path.join(os.path.dirname(__file__), 'assets', 'test_contract.json'),
    wasm_file=os.path.join(os.path.dirname(__file__), 'assets', 'test_contract.wasm'),
    substrate=substrate
)

contract = code.deploy(
    keypair=keypair,
    endowment=0,
    gas_limit=1000000000000,
    constructor="new",
    args={'init_value': True},
    upload_code=True
)

print(f'Deployed @ {contract.contract_address}')
```

**\*\*Work with an existing instance.\*\***

.. code:: python

```

# Create contract instance from deterministic address
contract = ContractInstance.create_from_address(
    contract_address=contract_address,
    metadata_file=os.path.join(os.path.dirname(__file__), 'assets', 'test_contract.json'),
    substrate=substrate
)

**Read data from a contract:**

.. code:: python

result = contract.read(keypair, 'get')
print('Current value of "get":', result.contract_result_data)

**Execute a contract call**

.. code:: python

# Do a gas estimation of the message
gas_predit_result = contract.read(keypair, 'flip')

print('Result of dry-run: ', gas_predit_result.value)
print('Gas estimate: ', gas_predit_result.gas_required)

# Do the actual call
print('Executing contract call...')
contract_receipt = contract.exec(keypair, 'flip', args={

}, gas_limit=gas_predit_result.gas_required)

if contract_receipt.is_success:
    print(f'Events triggered in contract: {contract_receipt.contract_events}')
else:
    print(f'Error message: {contract_receipt.error_message}')

sdk\interface package
=====

```

## Subpackages

-----

```
.. toctree::  
    :maxdepth: 4
```

    sdk\_interface.utils

## Submodules

-----

    sdk\interface.base module

-----

```
.. automodule:: sdk_interface.base  
    :members:  
    :undoc-members:  
    :show-inheritance:
```

    sdk\interface.constants module

-----

```
.. automodule:: sdk_interface.constants  
    :members:  
    :undoc-members:  
    :show-inheritance:
```

    sdk\interface.contracts module

-----

```
.. automodule:: sdk_interface.contracts  
    :members:  
    :undoc-members:  
    :show-inheritance:
```

    sdk\interface.exceptions module

-----

```
.. automodule:: sdk_interface.exceptions  
    :members:  
    :undoc-members:  
    :show-inheritance:
```

    sdk\interface.extensions module

---

```
.. automodule:: sdk_interface.extensions
    :members:
    :undoc-members:
    :show-inheritance:
```

sdk\\_interface.interfaces module

---

```
.. automodule:: sdk_interface.interfaces
    :members:
    :undoc-members:
    :show-inheritance:
```

sdk\\_interface.key module

---

```
.. automodule:: sdk_interface.key
    :members:
    :undoc-members:
    :show-inheritance:
```

sdk\\_interface.keypair module

---

```
.. automodule:: sdk_interface.keypair
    :members:
    :undoc-members:
    :show-inheritance:
```

sdk\\_interface.storage module

---

```
.. automodule:: sdk_interface.storage
    :members:
    :undoc-members:
    :show-inheritance:
```

Module contents

---

```
.. automodule:: sdk_interface
    :members:
```

```
:undoc-members:
:show-inheritance:
SDK Extension
=====
```

```
.. toctree::
    :maxdepth: 4
```

This extensions is meant as a fallback option that uses only existing Portaldot RPC methods. However, it is important to note that this fallback implementation is significantly inefficient, and it is encouraged to utilize third-party search indices where possible for optimal search performance.

-----

**\*\*Initialization\*\***

```
.. code:: python
```

```
from substrateinterface import SubstrateNodeExtension
```

```
portaldot = SubstrateInterface(
    url="wss://mainnet.portaldot.io",
    ss58_format=42,
    type_registry_preset='default'
)
```

```
# Provide maximum block range (bigger range descreases performance)
portaldot.register_extension(SubstrateNodeExtension(max_block_range=100))
```

**\*\*Implemented extension calls\*\***

```
.. code:: python
```

```
# filter_events
# Returns all `Balances.Transfer` events in block range #3 until #6
events = portaldot.extensions.filter_events(pallet_name="Balances", event_name="Transfer",
block_start=3, block_end=6)
```

```
.. code:: python
```



```

# filter_extrinsics
# All TransferKeepAlive extrinsics in block range #3 until #6
extrinsics = portaldot.extensions.filter_extrinsics(pallet_name="Timestamp",
call_name="transfer_keep_alive", block_start=3, block_end=6)

**Search block number**

.. code:: python

# Search for block number corresponding a specific datetime
block_datetime = datetime(2020, 7, 12, 0, 0, 0)

block_number = portaldot.extensions.search_block_number(block_datetime=block_datetime)

**Get block timestamp**

.. code:: python

# Get timestamp for specific block number
block_timestamp = portaldot.extensions.get_block_timestamp(block_number)

```

## Module Interface

### Contents:

- Constants
  - babe
  - balances
  - bounties
  - contracts
  - electionProviderMultiPhase
  - grandpa
  - identity

- indices
  - lottery
  - multisig
  - proxy
  - staking
  - system
  - timestamp
  - transactionPayment
  - treasury
  - vesting
- Errors
  - assets
  - babe
  - balances
  - bounties
  - contracts
  - electionProviderMultiPhase
  - grandpa
  - identity
  - imOnline
  - indices
  - lottery
  - multisig
  - proxy
  - scheduler
  - session
  - staking
  - system
  - transactionStorage
  - treasury
  - vesting

- Events
  - assets
  - balances
  - bounties
  - contracts
  - electionProviderMultiPhase
  - grandpa
  - identity
  - imOnline
  - indices
  - lottery
  - multisig
  - offences
  - proxy
  - scheduler
  - session
  - staking
  - system
  - transactionPayment
  - transactionStorage
  - treasury
  - utility
  - vesting
- Extrinsic
  - assets
  - babe
  - balances
  - bounties
  - contracts
  - electionProviderMultiPhase
  - grandpa

- identity
  - imOnline
  - indices
  - lottery
  - multisig
  - proxy
  - scheduler
  - session
  - staking
  - system
  - timestamp
  - transactionStorage
  - treasury
  - utility
  - vesting
- JSON RPC
  - author
  - babe
  - chain
  - childstate
  - contracts
  - grandpa
  - mmr
  - offchain
  - payment
  - rpc
  - state
  - syncstate
  - system
- Storage
  - assets

- authorship
- babe
- balances
- bounties
- contracts
- electionProviderMultiPhase
- grandpa
- identity
- imOnline
- indices
- lottery
- mmr
- multisig
- offences
- proxy
- randomnessCollectiveFlip
- scheduler
- session
- staking
- substrate
- system
- timestamp
- transactionPayment
- transactionStorage
- treasury
- vesting

If any information regarding modules are required kindly ask me back i will give you the detailed page for each one you need